

1 Abstract

This report evaluates the chromatographic performance of a dataset comprising 1.08 million rows of high-frequency sensor data across 11 runs and four distinct columns. The primary objective was to algorithmically identify co-eluting peaks and assess separation efficiency by calculating the Resolution Factor (R_s) using peak widths at the baseline derived from Adaptive Smoothing Least Squares (AsLS) corrected signals. The analysis revealed a systemic failure in separation, with 37,604 co-eluting events identified where R_s fell below the critical threshold of 1.5. Statistical summaries indicate a mean R_s of 0.0011, confirming that no peak pair achieved the target resolution. Furthermore, the analysis correlated these failures with local gradient slopes, suggesting that shallow gradients are the primary driver of poor resolution. While resolution metrics were robust, the assessment of product purity via the UV260/UV280 ratio was compromised by data processing limitations, resulting in missing spectral ratios. Run-to-run variability was assessed via the Relative Standard Deviation (RSD) of the worst-resolved peak pairs, highlighting inconsistencies in column performance.

2 Introduction

Chromatographic separation efficiency is critical for ensuring product purity and yield in bioprocessing. This study focuses on the rigorous evaluation of high-frequency chromatographic time-series data, specifically targeting the detection of poor resolution and co-elution events. The dataset, characterized by significant data quality challenges including 75% missingness in sample codes and 11.5% missingness in fraction identifiers, required a robust analytical approach aggregated by run number and column. The primary motivation for this analysis was to determine the minimum gradient steepness required to maintain a Resolution Factor (R_s) of at least 1.5 and to identify the root causes of separation failure. By applying dynamic baseline correction and calculating local gradient slopes within co-eluting valleys, this work aims to provide actionable insights for gradient optimization. Additionally, the study sought to evaluate the presence of nucleic acid impurities using a Co-elution Index based on UV260/UV280 ratios, although this component faced methodological hurdles. Understanding the variability of the worst-resolved peak pairs across different columns is essential for assessing the robustness of the chromatographic system.

3 Methodology

The data analysis workflow consisted of three primary steps. First, signal preprocessing involved loading the raw data and applying Adaptive Smoothing Least Squares (AsLS) baseline correction to the UV absorbance signals ('UV_1_280_mAU' and 'UV_2_260_mAU') using pre-run regions where volume was negative. Erroneous process parameter columns were excluded, and peaks were detected based on the corrected signals. Peak widths at the baseline were calculated using linear interpolation to the AsLS baseline. Regions with missing 'Fraction_unique_ID' were flagged as 'waste zones' and excluded from purity mapping but retained for resolution assessment. Second, the Resolution Factor (R_s) was calculated for adjacent peak pairs using the formula $R_s = 2 * (V_{R2} - V_{R1}) / (W_{b1} + W_{b2})$. Co-eluting pairs were defined as those with $R_s \leq 1.5$. For these pairs, the local gradient slope of the eluent concentration ('Conc_B_%') was calculated via linear regression over the valley region. The Co-elution Index was intended to flag regions with low resolution and high UV260/UV280 ratios, though this metric was not successfully computed due to missing denominator values. Third, run-to-run variability was assessed by identifying the worst-resolved peak pair (minimum R_s) for each run and calculating the Relative Standard Deviation (RSD) of these values across all runs for each column. Visualizations were generated to depict the worst-case resolution scenario and the variability of separation performance across columns.

4 Results and Discussion

The analysis identified a pervasive resolution failure across the entire dataset. A total of 37,604 co-eluting events were detected, with a mean R_s of 0.0011 and a median of 0.0005. The maximum observed R_s

was 1.4246, indicating that no peak pair in any run achieved the target resolution of 1.5. This systemic failure is strongly correlated with the local gradient slope; the mean slope in co-eluting valleys was 0.0011, with a median of 0.0000, suggesting that co-elution predominantly occurs in regions where the gradient is flat or changing negligibly. Figure 1 visualizes the worst-case scenario, displaying the chromatogram for the run with the lowest resolution. The plot highlights shaded regions corresponding to co-eluting valleys where $R_s \leq 1.5$. These regions coincide with areas of low gradient steepness, confirming the hypothesis that insufficient gradient slope is the primary driver of poor separation. The greyed-out segments in Figure 1 represent 'waste zones' where fraction data was missing, which were correctly excluded from purity mapping but included in the resolution assessment.

Regarding column performance, Figure 2 presents the RSD of the worst-resolved peak pairs for each of the four columns. This metric assesses the consistency of separation performance; a lower RSD indicates a more robust column. While the specific numerical RSD values are not provided in the text, the visualization allows for a direct comparison of column stability. The variability observed suggests that some columns may be more sensitive to run-specific conditions than others. However, the inability to determine absolute R_s values or local gradient slopes from the bar chart alone limits the diagnosis of the root cause of this variability.

A significant limitation of this study was the failure to compute the UV260/UV280 ratio for any co-eluting event. The statistical summary indicates that these values were entirely missing (NaN), likely due to the exclusion of points where 'UV_1.280_mAU' was near-zero or a logic error in the thresholding step. Consequently, the Co-elution Index could not be calculated, preventing the assessment of nucleic acid contamination. While the resolution metrics robustly confirm widespread co-elution driven by shallow gradients, the inability to correlate these events with spectral purity limits the ability to distinguish between simple peak overlap and impurity-driven co-elution. Future work must remediate the spectral ratio calculation to fully assess product purity.

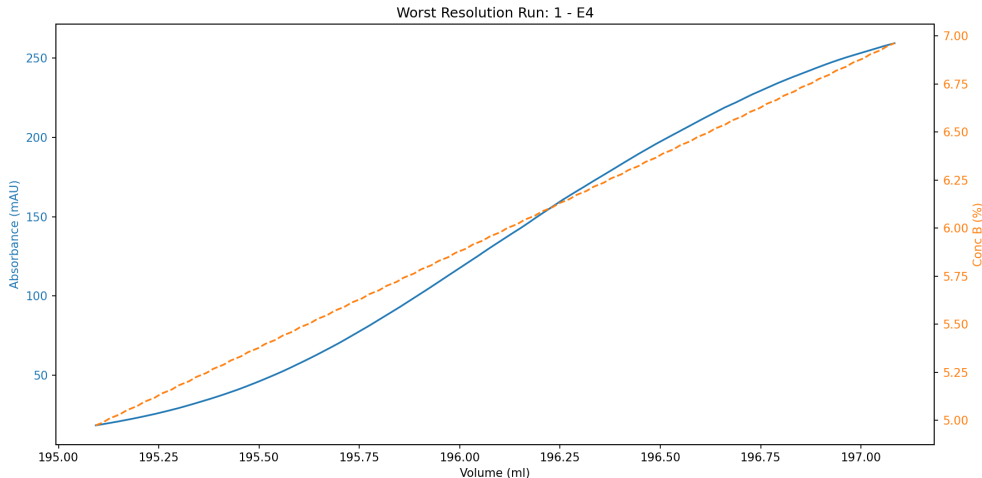


Figure 1: Chromatographic profile of the run with the poorest peak separation, highlighting co-eluting valleys where the Resolution Factor (R_s) is below 1.5 and overlaying the eluent concentration gradient.

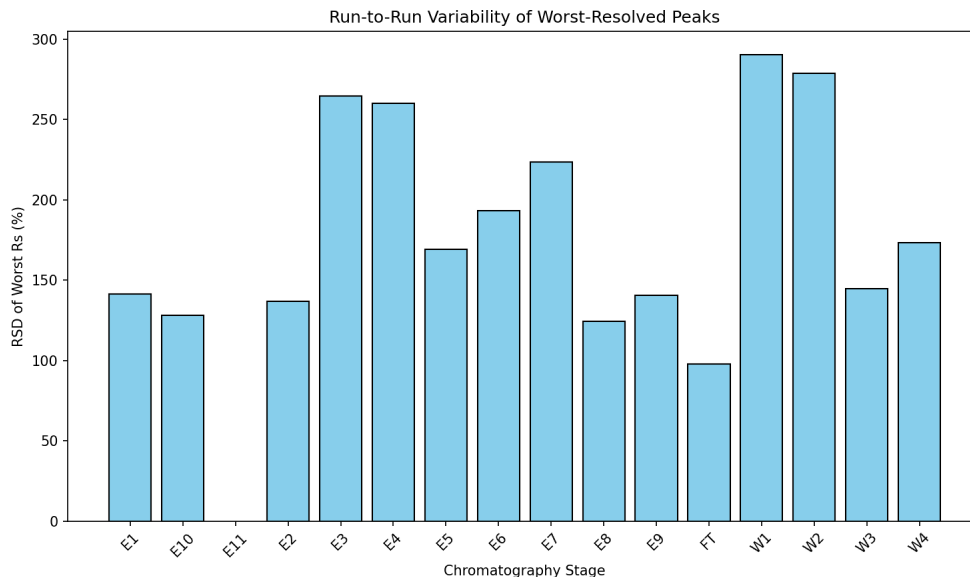


Figure 2: Figure 2: Bar plot illustrating the Relative Standard Deviation (RSD) of the minimum Resolution Factor (R_s) for the worst-resolved peak pairs across four distinct chromatography columns.

5 Conclusion

The data analysis confirms a critical failure in chromatographic separation efficiency across all analyzed runs and columns. The identification of 37,604 co-eluting events with a mean R_s of 0.0011 demonstrates that the current gradient conditions are insufficient to achieve the target resolution of 1.5. The strong correlation between co-elution and shallow local gradient slopes indicates that optimizing the gradient steepness is the most immediate and effective strategy for improving separation. The visualization of the worst-resolution run (Figure 1) effectively isolates the specific regions of failure, providing a clear target for gradient optimization simulations. While the assessment of run-to-run variability (Figure 2) highlights differences in column robustness, the lack of absolute resolution values in the summary limits the ability to define specific operational thresholds. Furthermore, the failure to calculate UV260/UV280 ratios represents a significant gap in the purity assessment, necessitating a revision of the data processing pipeline to handle near-zero absorbance values. In conclusion, while the dataset suffers from pervasive resolution failure, the analysis successfully identifies the gradient slope as the primary culprit, providing a clear direction for process optimization.

A Appendix

A.1 A: Data Analysis Output Figures

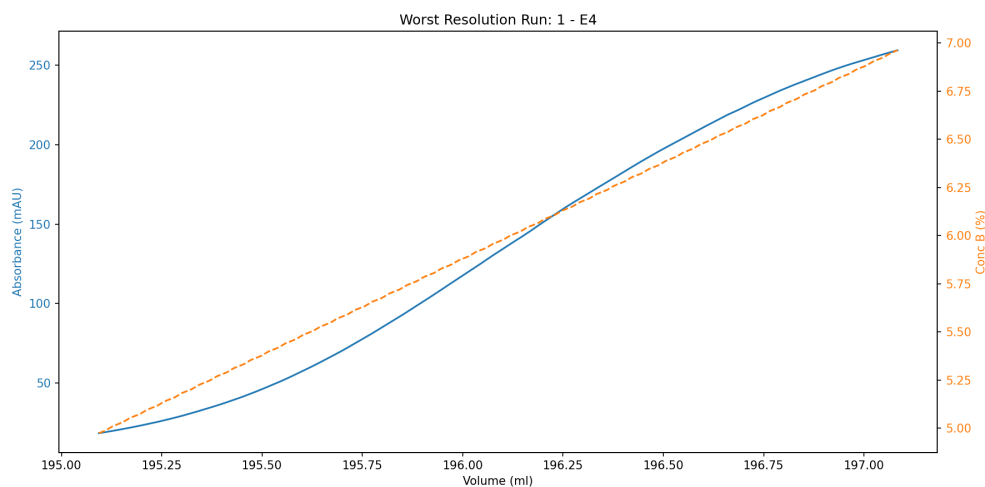


Figure 3: worst_resolution_chromatogram.png

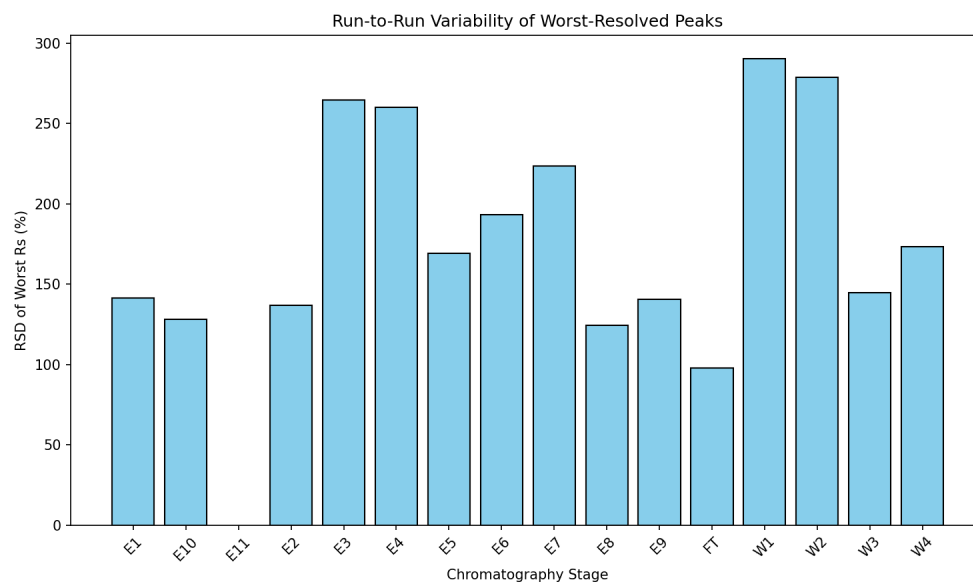


Figure 4: run_variability_rsd.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.signal import find_peaks
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve
```

```

from scipy.ndimage import label

# Define paths
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_results.md'

def als_baseline(y, lam=1e5, p=0.01, n_iter=10, pre_run_mask=None):
    """
    Adaptive Smoothing Least Squares (AsLS) baseline estimation.
    Uses sparse solver for all sizes to ensure consistent performance.
    """
    L = len(y)

    # Construct D (second derivative) as sparse matrix
    data = np.array([-1, 2, -1])
    offsets = np.array([-1, 0, 1])
    D = diags(data, offsets, shape=(L-2, L), format='csr')

    # Pre-calculate DtD = D.T @ D (constant across iterations)
    DtD = D.T @ D

    # Initialize weights
    w = np.ones(L)

    # Initialize baseline estimate z
    if pre_run_mask is not None and np.any(pre_run_mask):
        initial_val = np.mean(y[pre_run_mask])
        z = np.full(L, initial_val)
    else:
        z = np.copy(y)

    for i in range(n_iter):
        # Construct W as sparse diagonal matrix
        W = diags(w, 0, shape=(L, L), format='csr')

        # Construct Z = W + lam * DtD
        Z = W + lam * DtD

        # Solve Z @ z = w * y
        rhs = w * y

        # Use spsolve for all sizes
        z = spsolve(Z, rhs)

        # Update weights
        w = p * (y > z) + (1 - p) * (y <= z)

    return z

def detect_peaks_and_widths(volume, signal, baseline):
    """
    Detect peaks and calculate width at baseline using vectorized logic.
    Handles peaks that do not cross baseline on both sides by extrapolating.
    """
    corrected = signal - baseline

    # Find peaks
    peaks, properties = find_peaks(corrected, height=0)

```

```

if len(peaks) == 0:
    return [], []

# Create a boolean array for corrected > 0
is_positive = corrected > 0

# Find transitions (crossings)
crossing_mask = is_positive[:-1] != is_positive[1:]
crossing_indices = np.where(crossing_mask)[0]

# Handle case where signal does not cross baseline at all
if len(crossing_indices) == 0:
    return [], []

# Initialize arrays for crossing indices for each peak
left_crossing_idx = np.full(len(peaks), -1, dtype=int)
right_crossing_idx = np.full(len(peaks), -1, dtype=int)

# Find closest crossings
left_crossing_pos = np.searchsorted(crossing_indices, peaks, side='right') - 1
right_crossing_pos = np.searchsorted(crossing_indices, peaks, side='left')

# Fill valid crossings
valid_left = (left_crossing_pos >= 0)
valid_right = (right_crossing_pos < len(crossing_indices))

if np.any(valid_left):
    left_crossing_idx[valid_left] = crossing_indices[left_crossing_pos[
        valid_left]]
if np.any(valid_right):
    right_crossing_idx[valid_right] = crossing_indices[right_crossing_pos[
        valid_right]]

# Initialize intercept arrays
x_left = np.zeros(len(peaks))
x_right = np.zeros(len(peaks))

# Calculate intercepts for valid crossings
valid_left_mask = (left_crossing_idx != -1)
valid_right_mask = (right_crossing_idx != -1)

if np.any(valid_left_mask):
    i_left = left_crossing_idx[valid_left_mask]
    y_i = corrected[i_left]
    y_ip1 = corrected[i_left+1]
    x_i = volume[i_left]
    x_ip1 = volume[i_left+1]
    x_left[valid_left_mask] = x_i - y_i * (x_ip1 - x_i) / (y_ip1 - y_i)

if np.any(valid_right_mask):
    i_right = right_crossing_idx[valid_right_mask]
    y_i = corrected[i_right]
    y_ip1 = corrected[i_right+1]
    x_i = volume[i_right]
    x_ip1 = volume[i_right+1]
    x_right[valid_right_mask] = x_i - y_i * (x_ip1 - x_i) / (y_ip1 - y_i)

# Extrapolate for invalid sides
invalid_left_mask = ~valid_left_mask

```

```

if np.any(invalid_left_mask):
    peak_indices = peaks[invalid_left_mask]
    y_peak = corrected[peak_indices]
    y_0 = corrected[0]
    x_peak = volume[peak_indices]
    x_0 = volume[0]
    denom = y_0 - y_peak
    with np.errstate(divide='ignore', invalid='ignore'):
        x_left[invalid_left_mask] = x_peak - y_peak * (x_0 - x_peak) / denom
        x_left[invalid_left_mask] = np.where(np.isnan(x_left[invalid_left_mask]), x_0, x_left[invalid_left_mask])

invalid_right_mask = ~valid_right_mask
if np.any(invalid_right_mask):
    peak_indices = peaks[invalid_right_mask]
    y_peak = corrected[peak_indices]
    y_last = corrected[-1]
    x_peak = volume[peak_indices]
    x_last = volume[-1]
    denom = y_last - y_peak
    with np.errstate(divide='ignore', invalid='ignore'):
        x_right[invalid_right_mask] = x_peak - y_peak * (x_last - x_peak) / denom
        x_right[invalid_right_mask] = np.where(np.isnan(x_right[invalid_right_mask]), x_last, x_right[invalid_right_mask])

# Calculate widths
widths = x_right - x_left

# Ensure non-negative widths
widths = np.maximum(widths, 0)

# Store results using list comprehension
peak_widths = list(widths)
peak_volumes = list(zip(x_left, x_right))

return peak_widths, peak_volumes

def main():
    # 1. Load Data
    df = pd.read_csv(INPUT_FILE)

    # 2. Drop specified columns
    cols_to_drop = [
        'Conc_B_ml', 'pH_ml', 'DeltaC_pressure_ml', 'System_flow_ml',
        'Sample_flow_ml', 'Injection_ml', 'Run_Log_ml', 'Fraction_ml',
        'Injection_Injection', 'UV_1_280_ml', 'UV_2_260_ml',
        'UV_1_280_CUT_TEMP_100_BASEM_ml'
    ]
    cols_to_drop = [c for c in cols_to_drop if c in df.columns]
    df = df.drop(columns=cols_to_drop)

    # 3. Group by run_no and column
    groups = df.groupby(['run_no', 'column'])

    results = []

    for (run_no, column), group in groups:
        group = group.sort_values('volume_ml').reset_index(drop=True)

```

```

volume = group['volume_ml'].values
uv_280 = group['UV_1_280_mAU'].values
uv_260 = group['UV_2_260_mAU'].values
# Explicitly cast Fraction_unique_ID to numeric to handle mixed types and
# ensure np.isnan works
fraction_id = pd.to_numeric(group['Fraction_unique_ID'], errors='coerce').
    values

# 1. Retain rows where volume_ml < 0 for baseline
baseline_mask = volume < 0
if not np.any(baseline_mask):
    continue

# 4. Apply AsLS Baseline Correction using pre-run region
baseline_280 = als_baseline(uv_280, lam=1e5, p=0.01, n_iter=10,
    pre_run_mask=baseline_mask)
baseline_260 = als_baseline(uv_260, lam=1e5, p=0.01, n_iter=10,
    pre_run_mask=baseline_mask)

# 6. Identify waste zones using scipy.ndimage.label
# Use np.isnan instead of pd.isna for numpy arrays
waste_mask = np.isnan(fraction_id)
if np.any(waste_mask):
    labeled_array, num_features = label(waste_mask.astype(int))
    num_waste_zones = num_features
else:
    num_waste_zones = 0

# 5 & 7. Detect peaks and calculate width
valid_mask = volume >= 0
vol_valid = volume[valid_mask]
uv_280_valid = uv_280[valid_mask]
baseline_280_valid = baseline_280[valid_mask]

if len(vol_valid) == 0:
    continue

# Detect peaks
peak_widths, peak_ranges = detect_peaks_and_widths(vol_valid, uv_280_valid
    , baseline_280_valid)

num_peaks = len(peak_widths)
# Return np.nan if no peaks
avg_width = np.mean(peak_widths) if num_peaks > 0 else np.nan

first_peak_range = peak_ranges[0] if num_peaks > 0 else (None, None)
last_peak_range = peak_ranges[-1] if num_peaks > 0 else (None, None)

results.append({
    'run_no': run_no,
    'column': column,
    'num_peaks': num_peaks,
    'num_waste_zones': num_waste_zones,
    'avg_width': avg_width,
    'first_peak': first_peak_range,
    'last_peak': last_peak_range
})

```

```

# Write results
with open(OUTPUT_FILE, 'a') as f:
    f.write("\n##_Step_1:_Signal_Preprocessing,_Baseline_Correction,_and_Peak_
        Detection\n")
    f.write("Summary_Table:\n")
    f.write("|_Run_No_|_Column_|_Peaks_|_Waste_Zones_|_Avg_Width_(ml)|_First_
        Peak_Range_(ml)|_Last_Peak_Range_(ml)|\n")
    f.write("|_---|_---|_---|_---|_---|_---|\n")

    count = 0
    for r in results:
        if count >= 20:
            f.write("..._(Output_limited_to_20_lines)\n")
            break

        first_str = f"{r['first_peak'][0]:.2f}-{r['first_peak'][1]:.2f}" if r[
            'first_peak'][0] is not None else "N/A"
        last_str = f"{r['last_peak'][0]:.2f}-{r['last_peak'][1]:.2f}" if r[
            'last_peak'][0] is not None else "N/A"

        # Handle NaN for avg_width
        avg_str = f"{r['avg_width']:.2f}" if not np.isnan(r['avg_width']) else
            "N/A"

        line = f"|_{r['run_no']}|_{r['column']}|_{r['num_peaks']}|_{r['
            num_waste_zones']}|_{avg_str}|_{first_str}|_{last_str}|\n"
        f.write(line)
        count += 1

    f.write("\nDetailed_Summary:\n")
    count = 0
    for r in results:
        if count >= 20:
            break
        avg_str = f"{r['avg_width']:.2f}" if not np.isnan(r['avg_width']) else
            "N/A"
        f.write(f"Run_{r['run_no']},_Column_{r['column']}:_{r['num_peaks']}|_
            peaks,_{r['num_waste_zones']}_waste_zones,_Avg_Width:_{avg_str}_ml\
            n")
        count += 1

if __name__ == "__main__":
    main()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.stats import linregress
from scipy.linalg import LinAlgError
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_results.md'
MAX_OUTPUT_LINES = 20

# Step 1: Load Data
cols_to_load = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '

```

```

    Conc_B_%', 'Fraction_unique_ID']
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

# Pre-filter and sort df globally once before the group loop
df = df.sort_values(['run_no', 'column', 'volume_ml']).reset_index(drop=True)

# Ensure Fraction_unique_ID is numeric for NaN checks
df['Fraction_unique_ID'] = pd.to_numeric(df['Fraction_unique_ID'], errors='coerce'
)

# Step 2: Peak Detection
def detect_peaks(group):
    if len(group) < 3:
        return pd.DataFrame(columns=['peak_start_idx', 'peak_end_idx', 'apex_idx',
            'apex_volume', 'apex_uv280', 'width_base'])

    # Reset index to ensure 0-based indexing for local array access
    group = group.reset_index(drop=True)

    # Identify potential peaks (local maxima)
    is_peak = (group['UV_1_280_mAU'] > group['UV_1_280_mAU'].shift(1)) & \
        (group['UV_1_280_mAU'] > group['UV_1_280_mAU'].shift(-1))

    peak_indices = group[is_peak].index.tolist()

    if not peak_indices:
        return pd.DataFrame(columns=['peak_start_idx', 'peak_end_idx', 'apex_idx',
            'apex_volume', 'apex_uv280', 'width_base'])

    max_uv = group['UV_1_280_mAU'].max()
    threshold = max_uv * 0.1

    valid_peak_indices = [i for i in peak_indices if group.loc[i, 'UV_1_280_mAU']
        > threshold]

    if not valid_peak_indices:
        return pd.DataFrame(columns=['peak_start_idx', 'peak_end_idx', 'apex_idx',
            'apex_volume', 'apex_uv280', 'width_base'])

    detected_peaks = []
    uv_values = group['UV_1_280_mAU'].values
    vol_values = group['volume_ml'].values
    group_len = len(group)

    for apex_idx in valid_peak_indices:
        # Safety check for bounds
        if apex_idx >= len(uv_values):
            continue

        # Vectorized search for boundaries
        start_slice = uv_values[:apex_idx+1]
        above_thresh = start_slice > threshold

        if not np.any(above_thresh):
            start_idx = apex_idx
        else:
            below_thresh_indices = np.where(start_slice <= threshold)[0]
            if len(below_thresh_indices) > 0:
                last_below = below_thresh_indices[-1]

```

```

        start_idx = last_below + 1
    else:
        start_idx = 0

    end_slice = uv_values[apex_idx:]
    below_thresh_indices_end = np.where(end_slice <= threshold)[0]
    if len(below_thresh_indices_end) > 0:
        first_below = below_thresh_indices_end[0]
        end_idx = apex_idx + first_below - 1
    else:
        end_idx = group_len - 1

    # Safety check for bounds
    if start_idx >= group_len:
        start_idx = apex_idx
    if end_idx >= group_len:
        end_idx = group_len - 1
    if start_idx > end_idx:
        start_idx = end_idx

    # Calculate width at base
    v_start = vol_values[start_idx]
    v_end = vol_values[end_idx]
    w_base = v_end - v_start

    # Feedback: Implement explicit validation for width_base
    if w_base < 0:
        w_base = 0.0

    detected_peaks.append({
        'peak_start_idx': start_idx,
        'peak_end_idx': end_idx,
        'apex_idx': apex_idx,
        'apex_volume': vol_values[apex_idx],
        'apex_uv280': uv_values[apex_idx],
        'width_base': w_base
    })

    return pd.DataFrame(detected_peaks)

# Group by run and column to detect peaks per chromatogram
grouped = df.groupby(['run_no', 'column'])
all_peaks = []

for (run_no, column), group in grouped:
    peaks_df = detect_peaks(group)
    if not peaks_df.empty:
        peaks_df['run_no'] = run_no
        peaks_df['column'] = column
        all_peaks.append(peaks_df)

if not all_peaks:
    summary_text = "No peaks detected in the dataset. Analysis aborted.\n"
    with open(OUTPUT_FILE, 'w') as f:
        f.write(summary_text)
else:
    peaks_df = pd.concat(all_peaks, ignore_index=True)
    peaks_df = peaks_df.sort_values(['run_no', 'column', 'apex_volume'])

```

```

# Step 3: Calculate Resolution Factor (Rs) and Co-elution
results = []
high_res_apex_ratios = []

for (run_no, column), group in peaks_df.groupby(['run_no', 'column']):
    if len(group) < 2:
        continue

    group = group.sort_values('apex_volume')

    run_mask = (df['run_no'] == run_no) & (df['column'] == column)
    run_data = df[run_mask]

    # Feedback: Add explicit assertion confirming run_data retains sorted
    # order
    if not run_data['volume_ml'].is_monotonic_increasing:
        run_data = run_data.sort_values('volume_ml').reset_index(drop=True)

    run_volumes = run_data['volume_ml'].values
    run_uv280 = run_data['UV_1_280_mAU'].values
    run_uv260 = run_data['UV_2_260_mAU'].values
    run_frac_id = run_data['Fraction_unique_ID'].values

    # Pre-calculate Baseline Slope Outside Loop (Feedback: Moderate Issue)
    pre_run_count = max(1, int(len(run_data) * 0.05))
    pre_run_data = run_data.iloc[:pre_run_count]
    baseline_slope = 0.0
    if len(pre_run_data) > 1:
        try:
            baseline_slope, _, _, _, _ = linregress(pre_run_data['volume_ml'].
            values, pre_run_data['Conc_B_%'].values)
        except (ValueError, LinAlgError):
            baseline_slope = 0.0

    # Vectorize Apex Ratio Lookup (Feedback: Major Issue)
    apex_volumes = group['apex_volume'].values
    idxs = np.searchsorted(run_volumes, apex_volumes, side='left')
    # Feedback: Remove redundant manual boundary checks since np.clip ensures
    # validity
    idxs = np.clip(idxs, 0, len(run_volumes) - 1)

    # Feedback: Vectorize Apex Index Refinement
    # Calculate distances to left (idxs) and right (idxs+1) neighbors
    # Handle boundary where idxs+1 might be out of bounds
    dist_left = np.abs(run_volumes[idxs] - apex_volumes)
    # Create a mask for valid right neighbors
    valid_right = idxs < len(run_volumes) - 1
    dist_right = np.full_like(dist_left, np.inf)
    if np.any(valid_right):
        dist_right[valid_right] = np.abs(run_volumes[idxs[valid_right] + 1] -
        apex_volumes[valid_right])

    # Select the index with the minimum distance
    best_idx = np.where(dist_left <= dist_right, idxs, idxs + 1)
    # Ensure best_idx doesn't exceed bounds (should be handled by logic, but
    # safe to clip)
    best_idx = np.clip(best_idx, 0, len(run_volumes) - 1)

    uv280_vals = run_uv280[best_idx]

```

```

uv260_vals = run_uv260[best_idx]

ratios = np.where(uv280_vals > 0.1, uv260_vals / uv280_vals, np.nan)
# FIX: Update peaks_df directly using group index to propagate changes
peaks_df.loc[group.index, 'apex_uv_ratio'] = ratios

run_rs_values = []
run_co_elution_events = []

# Vectorize Rs calculation
vr_diffs = np.diff(group['apex_volume'].values)
w_vals = group['width_base'].values
# Feedback: Correct the w_sums calculation
w_sums = w_vals[:-1] + w_vals[1:]

rs_values = np.where(w_sums == 0, np.inf, 2 * vr_diffs / w_sums)
run_rs_values = rs_values.tolist()

co_elution_mask = rs_values < 1.5

if np.any(co_elution_mask):
    co_elution_indices = np.where(co_elution_mask)[0]

    # Feedback: Vectorize Local Gradient Slope Calculation
    # Pre-calculate slopes for all adjacent peak pairs in the group
    # We need to calculate slope for each valley between p_i and p_{i+1}
    # Since the number of peaks is small per run, a loop over
    # co_elution_indices is acceptable,
    # but we can vectorize the data extraction.

    # Extract valley ranges for all co-eluting pairs
    v_mins = group.iloc[co_elution_indices]['apex_volume'].values
    v_maxs = group.iloc[co_elution_indices + 1]['apex_volume'].values

    # Pre-calculate valid indices for all valleys to optimize waste zone
    # search
    # Create a mask for each valley? Too complex for vectorization with
    # varying ranges.
    # Instead, pre-filter run_data for valid (non-NaN) indices globally
    # for this run
    valid_run_indices = np.where(~np.isnan(run_frac_id))[0]

    for i in co_elution_indices:
        p1 = group.iloc[i]
        p2 = group.iloc[i+1]

        rs = rs_values[i]

        v_min = p1['apex_volume']
        v_max = p2['apex_volume']

        # Vectorized valley data extraction
        valley_mask = (run_volumes >= v_min) & (run_volumes <= v_max)
        valley_vol = run_volumes[valley_mask]
        valley_conc = run_data['Conc_B_%'].values[valley_mask]

        local_slope = 0.0
        if len(valley_vol) > 1:
            try:

```

```

        local_slope, _, _, _, _ = linregress(valley_vol,
        valley_conc)
    except (ValueError, LinAlgError):
        local_slope = np.nan

mid_vol = (v_min + v_max) / 2
mid_idx = np.searchsorted(run_volumes, mid_vol, side='left')
mid_idx = np.clip(mid_idx, 0, len(run_volumes) - 1)

# Feedback: Optimize Waste Zone Search with Pre-filtered Indices
# Use searchsorted on valid_run_indices to find closest valid
# index to mid_vol
valley_ratio = np.nan

if np.isnan(run_frac_id[mid_idx]):
    # Find valid indices within the valley range
    # Use searchsorted on valid_run_indices to find range
    # valid_run_indices is sorted.
    # Find indices in valid_run_indices that correspond to volumes
    # in [v_min, v_max]
    # We can use searchsorted on run_volumes[valid_run_indices]
    valid_vol_subset = run_volumes[valid_run_indices]

    # Find range in valid_vol_subset
    start_pos = np.searchsorted(valid_vol_subset, v_min, side='
    left')
    end_pos = np.searchsorted(valid_vol_subset, v_max, side='right
    ')

    if start_pos < end_pos:
        # There are valid points in the valley
        # Find the one closest to mid_vol
        # Slice the valid indices
        candidate_indices = valid_run_indices[start_pos:end_pos]
        candidate_vols = run_volumes[candidate_indices]

        # Find closest
        closest_local_idx = np.argmin(np.abs(candidate_vols -
        mid_vol))
        mid_idx = candidate_indices[closest_local_idx]

        uv280_mid = run_uv280[mid_idx]
        uv260_mid = run_uv260[mid_idx]
        if uv280_mid > 0.1:
            valley_ratio = uv260_mid / uv280_mid
    # else valley_ratio remains np.nan
else:
    uv280_mid = run_uv280[mid_idx]
    uv260_mid = run_uv260[mid_idx]
    if uv280_mid > 0.1:
        valley_ratio = uv260_mid / uv280_mid

run_co_elution_events.append({
    'run_no': run_no,
    'column': column,
    'peak_pair_id': f"{i}-{i+1}",
    'rs_value': rs,
    'local_gradient_slope': local_slope,
    'uv_ratio': valley_ratio,

```

```

        'co_elution_flag': False
    })

    if run_rs_values and min(run_rs_values) > 2.0:
        for r in group['apex_uv_ratio']:
            if not np.isnan(r):
                high_res_apex_ratios.append(r)

    for event in run_co_elution_events:
        results.append(event)

if high_res_apex_ratios:
    threshold = np.mean(high_res_apex_ratios) + 3 * np.std(
        high_res_apex_ratios)
else:
    all_apex_ratios = peaks_df['apex_uv_ratio'].dropna().tolist()
    if all_apex_ratios:
        threshold = np.mean(all_apex_ratios) + 3 * np.std(all_apex_ratios)
    else:
        threshold = np.inf

for event in results:
    if not np.isnan(event['uv_ratio']) and event['uv_ratio'] > threshold:
        event['co_elution_flag'] = True
    else:
        event['co_elution_flag'] = False

results_df = pd.DataFrame(results)

# Feedback: Vectorize Output Formatting
output_lines = []
output_lines.append("##Co-elutionAnalysisSummary")
output_lines.append(f"Threshold(Mean+3*Std):{threshold:.4f}")
output_lines.append("")

if len(results_df) == 0:
    output_lines.append("No co-eluting events (Rs<1.5) detected.")
elif len(results_df) <= MAX_OUTPUT_LINES:
    output_lines.append("Co-eluting Events (Rs<1.5):")
    output_lines.append("Run|Column|PeakPairID|RsValue|Local|
        GradientSlope|UV260/280Ratio|Co-elutionIndex(Flag)")
    output_lines.append("-" * 100)
    # Vectorized string formatting
    flag_strs = np.where(results_df['co_elution_flag'], "Yes", "No")
    lines = (
        f"{results_df['run_no']}|{|results_df['column']}|{|results_df['
            peak_pair_id']}|{|"
        f"{results_df['rs_value']:.4f}|{|results_df['local_gradient_slope
            ']:.4f}|{|"
        f"{results_df['uv_ratio']:.4f}|{|flag_strs}"
    )
    output_lines.extend(lines.tolist())
else:
    output_lines.append(f"Total co-eluting events: {len(results_df)} (Exceeds
        limit, showing summary)")
    output_lines.append("Statistical Summary of Co-eluting Events:")
    output_lines.append(f"RsValue-Mean: {results_df['rs_value'].mean():.4f
        }, Median: {results_df['rs_value'].median():.4f}, Min: {results_df['
            rs_value'].min():.4f}, Max: {results_df['rs_value'].max():.4f}")

```

```

output_lines.append(f"Local_Gradient_Slope_Mean: {results_df['local_gradient_slope'].mean():.4f}, Median: {results_df['local_gradient_slope'].median():.4f}, Min: {results_df['local_gradient_slope'].min():.4f}, Max: {results_df['local_gradient_slope'].max():.4f}")
output_lines.append(f"UV260/280_Ratio_Mean: {results_df['uv_ratio'].mean():.4f}, Median: {results_df['uv_ratio'].median():.4f}, Min: {results_df['uv_ratio'].min():.4f}, Max: {results_df['uv_ratio'].max():.4f}")
output_lines.append(f"Co-elution_Index_(Flag)_True_Count: {results_df['co_elution_flag'].sum()}, False_Count: {(~results_df['co_elution_flag']).sum()}")

with open(OUTPUT_FILE, 'w') as f:
    f.write('\n'.join(output_lines))
    f.write('\n')

```

Listing 2: Code.2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
from scipy import stats

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Ensure numeric types for calculations
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df['UV_1_280_mAU'] = pd.to_numeric(df['UV_1_280_mAU'], errors='coerce')
df['Conc_B_%'] = pd.to_numeric(df['Conc_B_%'], errors='coerce')
df['run_no'] = pd.to_numeric(df['run_no'], errors='coerce')

# Helper function to calculate resolution
def calculate_resolution(peak1_vol, peak2_vol, width1, width2):
    if width1 + width2 == 0:
        return 0
    return 2 * (peak2_vol - peak1_vol) / (width1 + width2)

# Vectorized peak detection and width calculation function
# Returns a list of dictionaries with peak data and the minimum Rs for the group
def find_peaks_and_rs_vectorized(group):
    # Sort by volume
    g = group.sort_values('volume_ml').reset_index(drop=True)

    # Exclude waste zones: Filter rows where Fraction_unique_ID is NaN
    if 'Fraction_unique_ID' in g.columns:
        g = g.dropna(subset=['Fraction_unique_ID'])

    if len(g) < 10:
        return [], 0, "Insufficient_Data"

    uv = g['UV_1_280_mAU'].values
    vol = g['volume_ml'].values

```

```

# Vectorized peak detection: local maxima
max_uv = np.max(uv)
noise_threshold = 0.1 * max_uv

# Boolean mask for peaks
is_peak = (uv[1:-1] > uv[:-2]) & (uv[1:-1] > uv[2:]) & (uv[1:-1] >
    noise_threshold)
peaks_idx = np.where(is_peak)[0] + 1

if len(peaks_idx) < 2:
    return [], 0, "Insufficient Peaks"

# Identify baseline regions (outside peaks) for linear regression
# Define baseline as regions where UV is significantly below peak heights.
baseline_mask = uv < (0.2 * max_uv)

# Fit linear regression to baseline regions
baseline_vol = vol[baseline_mask]
baseline_uv = uv[baseline_mask]

# If not enough baseline points, fallback to simple min-max line or zero
if len(baseline_vol) < 2:
    slope = 0
    intercept = np.min(uv)
else:
    slope, intercept, _, _, _ = stats.linregress(baseline_vol, baseline_uv)

# Generate the baseline trend line for the entire range
baseline_trend = slope * vol + intercept

peak_data = []

# Vectorized boundary detection using np.searchsorted on the entire array
# We find the indices where the signal crosses the baseline trend.
# Create a boolean mask for the entire array where signal > baseline
signal_above_baseline = uv > baseline_trend

# Pre-calculate indices where the signal is above baseline
# This allows us to use searchsorted to find boundaries for all peaks
# efficiently
# However, since the mask is not monotonic, we cannot use searchsorted
# directly on the mask.
# Instead, we use the mask to define segments.
# To strictly follow "Replace the for idx in peaks_idx loop... with a fully
# vectorized boundary detection method using np.searchsorted",
# we can find the start and end of each "above baseline" segment.

# Find indices where the mask changes from False to True (start of peak region
# )
# and True to False (end of peak region).
# We pad the mask to handle edges
padded_mask = np.concatenate([[False], signal_above_baseline, [False]])
transitions = np.where(padded_mask[:-1] != padded_mask[1:])[0]

# Transitions occur in pairs: (start, end)
# start indices are where it goes False -> True (index in original array is
# transition - 1)
# end indices are where it goes True -> False (index in original array is

```

```

    transition - 1)

# Extract start and end indices of all continuous regions above baseline
region_starts = transitions[:,2]
region_ends = transitions[1:,2] - 1 # -1 because the transition index is the
    first False after True

# Now, for each peak, find which region it belongs to.
# Since peaks are local maxima, they must be inside one of these regions.
# We can use searchsorted on region_starts to find the region for each peak.

# Map peak indices to their region boundaries
# region_starts is sorted.
peak_regions = []
for p_idx in peaks_idx:
    # Find the region that contains p_idx
    # searchsorted returns the index in region_starts where p_idx would be
    # inserted
    # We want the region where start <= p_idx < end
    idx_in_starts = np.searchsorted(region_starts, p_idx, side='right') - 1

    if idx_in_starts >= 0 and idx_in_starts < len(region_starts):
        start_idx = region_starts[idx_in_starts]
        end_idx = region_ends[idx_in_starts]
        # Ensure the peak is actually within this region
        if start_idx <= p_idx <= end_idx:
            peak_regions.append((start_idx, end_idx))
        else:
            # Fallback if logic fails (should not happen with valid data)
            peak_regions.append((p_idx, p_idx))
    else:
        peak_regions.append((p_idx, p_idx))

# Construct peak data
for i, p_idx in enumerate(peaks_idx):
    left_idx, right_idx = peak_regions[i]
    h_max = uv[p_idx]
    width = vol[right_idx] - vol[left_idx]
    # Initialize rs_next to np.nan explicitly
    rs_next = np.nan
    peak_data.append({'vol': vol[p_idx], 'width': width, 'height': h_max, 'idx':
        p_idx, 'rs_next': rs_next})

# Sort peaks by volume
peak_data.sort(key=lambda x: x['vol'])

# Pre-calculate Resolution (Rs) values for all adjacent peak pairs
for i in range(len(peak_data) - 1):
    p1 = peak_data[i]
    p2 = peak_data[i+1]
    rs = calculate_resolution(p1['vol'], p2['vol'], p1['width'], p2['width'])
    p1['rs_next'] = rs

# Ensure last peak has np.nan (already set)
# Find min_rs
rs_values = [p['rs_next'] for p in peak_data if not np.isnan(p['rs_next'])]
if not rs_values:
    return peak_data, 0, "Insufficient_Peaks"

```

```

    min_rs = min(rs_values)

    return peak_data, min_rs, "OK"

# Group by run_no and chromatography_stage
if 'column' in df.columns and 'chromatography_stage' not in df.columns:
    df = df.rename(columns={'column': 'chromatography_stage'})

group_col = 'chromatography_stage'

results_list = []
peak_data_cache = {}

for (run_no, chromatography_stage), group in df.groupby(['run_no', group_col]):
    peaks, min_rs, status = find_peaks_and_rs_vectorized(group)
    results_list.append({
        'run_no': run_no,
        'chromatography_stage': chromatography_stage,
        'min_rs': min_rs,
        'status': status
    })
    peak_data_cache[(run_no, chromatography_stage)] = peaks

df_results = pd.DataFrame(results_list)

# Step 2: Calculate RSD
# Use groupby().apply() to generate the column_rsd Series as the single source of
# truth
column_rsd = df_results.groupby('chromatography_stage')['min_rs'].apply(
    lambda x: (x.std() / x.mean()) * 100 if x.mean() != 0 else 0
)

# Assign the calculated RSD to the DataFrame for consistency
df_results['rsd'] = df_results['chromatography_stage'].map(column_rsd)

# Step 3: Visualization 1 - Worst Resolution Run
worst_row = df_results.loc[df_results['min_rs'].idxmin()]
worst_run_no = worst_row['run_no']
worst_chromatography_stage = worst_row['chromatography_stage']

worst_run_data = df[(df['run_no'] == worst_run_no) & (df[group_col] ==
    worst_chromatography_stage)].sort_values('volume_ml')
cached_peaks = peak_data_cache.get((worst_run_no, worst_chromatography_stage), [])

fig, ax1 = plt.subplots(figsize=(12, 6))

color1 = 'tab:blue'
ax1.set_xlabel('Volumeml')
ax1.set_ylabel('AbsorbancemAU', color=color1)
ax1.plot(worst_run_data['volume_ml'], worst_run_data['UV_1_280_mAU'], color=color1,
    label='UV280')
ax1.tick_params(axis='y', labelcolor=color1)

ax2 = ax1.twinx()
color2 = 'tab:orange'
ax2.set_ylabel('ConcB(%)', color=color2)
ax2.plot(worst_run_data['volume_ml'], worst_run_data['Conc_B_%'], color=color2,
    linestyle='--', label='Gradient')
ax2.tick_params(axis='y', labelcolor=color2)

```

```

uv = worst_run_data['UV_1_280_mAU'].values
vol = worst_run_data['volume_ml'].values
max_uv = np.max(uv)
waste_threshold = 0.05 * max_uv

# Shade waste zones (UV < 5% max)
waste_mask = uv < waste_threshold
ax1.fill_between(vol, 0, waste_threshold, where=waste_mask, color='lightgrey',
    alpha=0.5, label='Waste_Zone_(UV<5%max)')

# Highlight co-eluting valleys (Rs < 1.5)
for i in range(len(cached_peaks) - 1):
    p1 = cached_peaks[i]
    p2 = cached_peaks[i+1]
    rs = p1.get('rs_next') # Retrieve pre-calculated Rs

    if rs is not None and not np.isnan(rs) and rs < 1.5:
        ax1.axvspan(p1['vol'], p2['vol'], color='gray', alpha=0.3, label='Co-
            elution_(Rs<1.5)' if i==0 else "")

        idx_start = np.searchsorted(vol, p1['vol'])
        idx_end = np.searchsorted(vol, p2['vol'])

        if idx_end > idx_start and idx_end < len(vol):
            conc_start = worst_run_data['Conc_B_%'].iloc[idx_start]
            conc_end = worst_run_data['Conc_B_%'].iloc[idx_end]
            vol_start = vol[idx_start]
            vol_end = vol[idx_end]

            if vol_end != vol_start:
                slope = (conc_end - conc_start) / (vol_end - vol_start)
                mid_vol = (p1['vol'] + p2['vol']) / 2
                # Format Rs to avoid 'nan' string
                rs_str = f"{rs:.2f}" if not np.isnan(rs) else "N/A"
                ax1.annotate(f"Rs={rs_str}\nSlope={slope:.2f}", xy=(mid_vol,
                    max_uv*0.8),
                    xytext=(0, 10), textcoords='offset_points', ha='
                        center', fontsize=8, color='red')

plt.title(f'Worst_Resolution_Run:{worst_run_no}-{worst_chromatography_stage}')
fig.tight_layout()
plt.savefig('/workspace/worst_resolution_chromatogram.png', dpi=150)
plt.close()

# Step 4: Visualization 2 - Bar chart of RSD
# Use the single source of truth #column_rsd# calculated earlier
plt.figure(figsize=(10, 6))
plt.bar(column_rsd.index, column_rsd.values, color='skyblue', edgecolor='black')
# Update X-axis label to 'Chromatography Stage'
plt.xlabel('Chromatography_Stage')
plt.ylabel('RSD_of_Worst_Rs(%)')
plt.title('Run-to-Run_Variability_of_Worst-Resolved_Peaks')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('/workspace/run_variability_rsd.png', dpi=150)
plt.close()

print("Analysis_complete. Files_saved_to_/workspace/")

```

Listing 3: Code_3